

Universidad Autónoma de Baja California

Facultad de Ingeniería, Mexicali



Microcontroladores

Actividad 4.8: Generador de Funciones (DAC-SPI)

Le Gresley Ferrel Andres Alejandro

Jimenez Acevedo Francisco Alejandro

Profesor:

Aguilar Avelar Carlos Alberto

3 de Mayo del 2026

Descripción

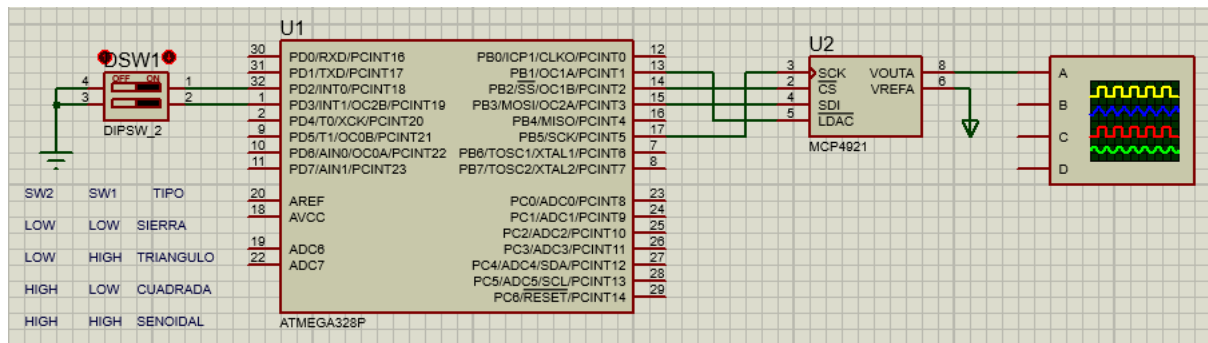
Utilizando las interfaces de comunicaciones seriales SPI o USPI del microcontrolador ATmega328P, diseñe un generador de funciones. Realice la simulación en Proteus utilizando el DAC MCP4921. Su diseño debe contar con las siguientes características:

Especificaciones:

- Permite generar señales cuadradas, triangulares, diente de sierra y senoidales.
- El tipo de señal generada se selecciona con interruptores deslizables.
- Las señales generadas tienen una frecuencia fija de 100 Hz
- Las señales generadas tienen una amplitud fija de 5 Vpp.

Diagrama de conexiones esquemático realizado

Se realizaron las conexiones entre el dip-switch de dos pines hacia el puerto D del microcontrolador ATmega328P y desde el puerto B del mismo hacia el DAC MCP4921 y el osciloscopio a la salida del DAC, obteniendo el siguiente diagrama de conexiones:



Implementación de código

Se realizó el código en C para programar las funciones del microcontrolador:

C/C++

```
/*
 * =====
 *  Generador de Funciones - ATmega328P + DAC MCP4921 (SPI)
 * =====
 *  Señales : Senoidal, Cuadrada, Triangular, Diente de Sierra
 *  Frecuencia : 100 Hz fija
 *  Amplitud   : 5 Vpp (0 - 4095 en DAC 12 bits, Vref = 5 V)
 *
 *  Selección de señal con DIP switches en PD2 y PD3:
 *    PD2=0, PD3=0 ? Senoidal
 *    PD2=1, PD3=0 ? Cuadrada
 *    PD2=0, PD3=1 ? Triangular
 *    PD2=1, PD3=1 ? Diente de sierra
 *
 *  Conexiones SPI:
 *    PB2 (SS/CS) ? CS del MCP4921
 *    PB3 (MOSI)  ? SDI del MCP4921
 *    PB5 (SCK)   ? SCK del MCP4921
 *    PB1         ? LDAC del MCP4921 (forzado en LOW para
 *                               actualización inmediata)
 *
 *  Temporización:
 *    F_CPU = 16 MHz
 *    Muestras por ciclo (N) = 100
 *    Timer0 (CTC) ? ISR cada 100 µs ? 100 Hz × 100 muestras
 */
```

```

* =====
*/

#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <math.h>
#include <stdint.h>

/* ??? Parámetros globales ??? */
#define N_SAMPLES 100 /* muestras por periodo */
#define DAC_MAX 4095 /* resolución 12 bits */
#define DAC_HALF 2048 /* punto medio */
#define PI 3.14159265f

/* ??? Pines de control ??? */
#define CS_DDR DDRB
#define CS_PORT PORTB
#define CS_PIN PB2 /* Chip Select MCP4921 */

#define LDAC_DDR DDRB
#define LDAC_PORT PORTB
#define LDAC_PIN PB1 /* LDAC (LOW = actualizar salida) */

#define SW_DDR DDRD
#define SW_PORT PORTD
#define SW_PIN_0 PD2 /* bit 0 selector de forma */
#define SW_PIN_1 PD3 /* bit 1 selector de forma */

/* ??? Tipos de onda ??? */
typedef enum {
    WAVE_SINE = 0,
    WAVE_SQUARE = 1,
    WAVE_TRIANGLE = 2,
    WAVE_SAWTOOTH = 3
} WaveType;

/* ??? Variables de tabla LUT ??? */
static uint16_t lut_sine[N_SAMPLES];

/* ??? Estado del generador ??? */
volatile uint8_t sample_index = 0;
volatile uint8_t new_sample = 0; /* bandera desde ISR Timer0 */

/* ??? Prototipos ??? */
void spi_init(void);
void timer0_init(void);

```

```

void    gpio_init(void);
void    lut_build(void);
uint16_t wave_sample(WaveType wt, uint8_t idx);
void    dac_write(uint16_t value);
WaveType read_switches(void);

/* =====
 *  PROGRAMA PRINCIPAL
 *  ===== */
int main(void)
{
    gpio_init();
    spi_init();
    timer0_init();
    lut_build();

    sei();    /* habilitar interrupciones globales */

    while (1)
    {
        if (new_sample)
        {
            new_sample = 0;

            WaveType wt = read_switches();
            uint16_t val = wave_sample(wt, sample_index);
            dac_write(val);
        }
    }
}

/* =====
 *  ISR Timer0 Compare Match A  (cada 100 µs ? 10 kHz)
 *  100 muestras × 100 µs = 10 ms = 100 Hz
 *  ===== */
ISR(TIMER0_COMPA_vect)
{
    sample_index++;
    if (sample_index >= N_SAMPLES)
        sample_index = 0;

    new_sample = 1;
}

/* =====
 *  Inicialización de GPIO
 *  ===== */
void gpio_init(void)

```

```

{
    /* CS y LDAC como salidas, inicialmente HIGH */
    CS_DDR   |= (1 << CS_PIN);
    CS_PORT  |= (1 << CS_PIN);

    LDAC_DDR |= (1 << LDAC_PIN);
    LDAC_PORT &= ~(1 << LDAC_PIN); /* LDAC fijo en LOW: actualización
inmediata */

    /* Switches como entradas con pull-up interno */
    SW_DDR &= ~((1 << SW_PIN_0) | (1 << SW_PIN_1));
    SW_PORT |= ((1 << SW_PIN_0) | (1 << SW_PIN_1));
}

/* =====
*   Inicialización SPI (Master, Modo 0,0 - CPOL=0, CPHA=0)
*   SCK = F_CPU / 16 = 1 MHz (SPR1=0, SPR0=1, SPI2X=0)
* ===== */
void spi_init(void)
{
    /* MOSI, SCK y SS como salidas */
    DDRB |= (1 << PB3) | (1 << PB5) | (1 << PB2);

    /* Activar SPI, modo Master, F_osc/16 */
    SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPR0);
    SPSR &= ~(1 << SPI2X);
}

/* =====
*   Inicialización Timer0 en modo CTC
*   OCR0A = (F_CPU / (prescaler * f_isr)) - 1
*   f_isr = 10 000 Hz (100 µs)
*   prescaler = 8 ? OCR0A = (16e6 / (8 * 10000)) - 1 = 199
* ===== */
void timer0_init(void)
{
    TCCR0A = (1 << WGM01);          /* CTC mode */
    TCCR0B = (1 << CS01);          /* prescaler /8 */
    OCR0A = 199;                   /* TOP = 199 */
    TIMSK0 = (1 << OCIE0A);        /* habilitar interrupción CTC */
}

/* =====
*   Construir LUT senoidal (evita float en la ISR)
* ===== */
void lut_build(void)
{
    for (uint8_t i = 0; i < N_SAMPLES; i++)

```

```

    {
        /* Seno normalizado 0..1, escalado a 0..4095 */
        float s = (sinf(2.0f * PI * i / N_SAMPLES) + 1.0f) / 2.0f;
        lut_sine[i] = (uint16_t)(s * DAC_MAX);
    }
}

/* =====
 *   Calcular muestra según forma de onda
 * ===== */
uint16_t wave_sample(WaveType wt, uint8_t idx)
{
    switch (wt)
    {
        /* ?? Senoidal ?? */
        case WAVE_SINE:
            return lut_sine[idx];

        /* ?? Cuadrada ?? */
        case WAVE_SQUARE:
            return (idx < N_SAMPLES / 2) ? DAC_MAX : 0;

        /* ?? Triangular ?? */
        case WAVE_TRIANGLE:
            if (idx < N_SAMPLES / 2)
                /* subida: 0 ? 4095 en N/2 pasos */
                return (uint16_t)((uint32_t)idx * DAC_MAX / (N_SAMPLES / 2));
            else
                /* bajada: 4095 ? 0 en N/2 pasos */
                return (uint16_t)((uint32_t)(N_SAMPLES - 1 - idx) * DAC_MAX /
(N_SAMPLES / 2));

        /* ?? Diente de sierra ?? */
        case WAVE_SAWTOOTH:
        default:
            return (uint16_t)((uint32_t)idx * DAC_MAX / (N_SAMPLES - 1));
    }
}

/* =====
 *   Leer DIP switches y retornar tipo de onda
 *   Switches activos en LOW (pull-up interno activado)
 *   SW0=0, SW1=0 ? SINE
 *   SW0=1, SW1=0 ? SQUARE
 *   SW0=0, SW1=1 ? TRIANGLE
 *   SW0=1, SW1=1 ? SAWTOOTH
 * ===== */
WaveType read_switches(void)

```

```

{
    uint8_t sw0 = !(PIND & (1 << SW_PIN_0)); /* 1 si presionado */
    uint8_t sw1 = !(PIND & (1 << SW_PIN_1));

    return (WaveType)((sw1 << 1) | sw0);
}

/* =====
 * Enviar valor de 12 bits al MCP4921 vía SPI
 *
 * Trama MCP4921 (16 bits):
 * Bit 15: A/B    = 0 (canal A)
 * Bit 14: BUF    = 0 (sin buffer en VREF)
 * Bit 13: GA     = 1 (ganancia 1x ? Vout = D/4096 × Vref)
 * Bit 12: SHDN   = 1 (salida activa)
 * Bits 11-0: datos (D11..D0)
 * ===== */
void dac_write(uint16_t value)
{
    /* Limitar a 12 bits */
    value &= 0x0FFF;

    /* Construir palabra de 16 bits:
     * A/B=0, BUF=0, GA=1, SHDN=1, D[11:0] */
    uint16_t word = 0x3000 | value;

    uint8_t high_byte = (uint8_t)(word >> 8);
    uint8_t low_byte  = (uint8_t)(word & 0xFF);

    /* Activar CS (LOW) */
    CS_PORT &= ~(1 << CS_PIN);

    /* Enviar byte alto */
    SPDR = high_byte;
    while (!(SPSR & (1 << SPIF)));

    /* Enviar byte bajo */
    SPDR = low_byte;
    while (!(SPSR & (1 << SPIF)));

    /* Desactivar CS (HIGH) ? latch del dato en MCP4921 */
    CS_PORT |= (1 << CS_PIN);

    /* LDAC ya está fijo en LOW ? la salida se actualiza inmediatamente */
}

```

[Enlace al video de demostración de la simulación y funcionamiento](#)

Conclusiones

Alejandro Jimenez: Esta actividad demostró la viabilidad de implementar un generador de funciones básico utilizando recursos limitados de un microcontrolador como el ATmega328P. A pesar de trabajar con una frecuencia y amplitud fijas, el sistema logra cumplir con los requerimientos esenciales mediante una correcta configuración del SPI y una adecuada programación de las tablas de datos para cada forma de onda. Se identificó que la precisión de la señal depende tanto de la resolución del DAC como del número de muestras utilizadas, lo cual abre la puerta a futuras mejoras. Asimismo, el diseño resalta la importancia de estructurar el código de manera eficiente para garantizar tiempos de actualización constantes, especialmente en señales como la senoidal. En conclusión, el proyecto no solo cumple con los objetivos planteados, sino que también sienta bases sólidas para el desarrollo de sistemas más complejos de generación y procesamiento de señales.

Andres Le Gresley: El desarrollo de este generador de funciones permitió comprender de manera práctica la integración entre hardware y software en sistemas embebidos. El uso de la comunicación SPI del ATmega328P para controlar el DAC MCP4921 evidenció la importancia de los protocolos seriales en la generación precisa de señales analógicas. A través de la implementación de distintas formas de onda (cuadrada, triangular, diente de sierra y senoidal), se reforzó el concepto de síntesis digital de señales y su conversión al dominio analógico. Además, la selección de señales mediante interruptores deslizables mostró una forma eficiente de interacción usuario-sistema. La simulación en Proteus facilitó la validación del diseño antes de una posible implementación física, destacando la relevancia de las herramientas de simulación en el proceso de ingeniería. En conjunto, la actividad consolidó conocimientos clave en microcontroladores, manejo de periféricos y generación de señales.